

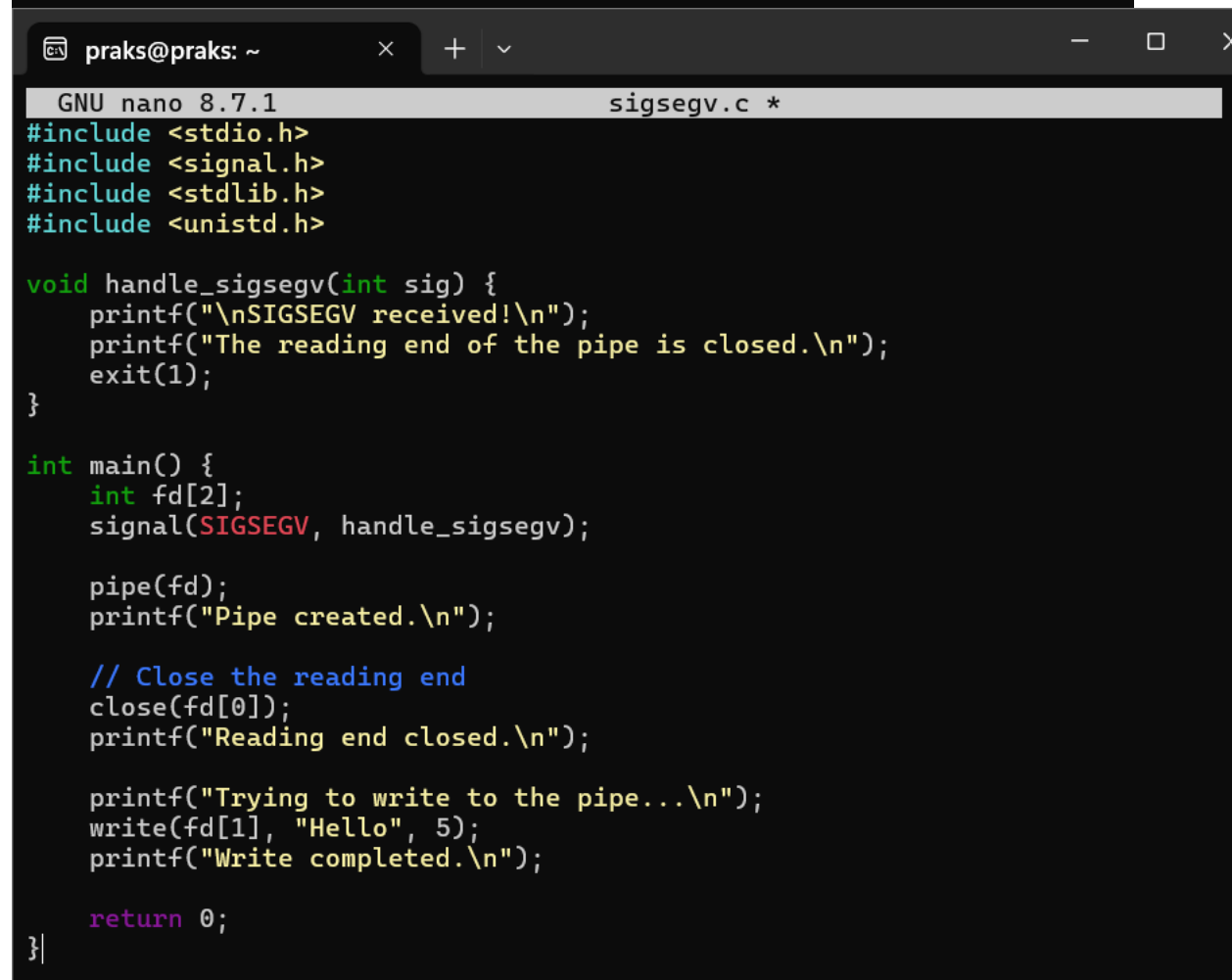
```
praks@praks: ~  
praks@praks:~$ nano signal1.c  
praks@praks:~$ gcc signal1.c -o signal1  
./signal1  
# Press Ctrl+C to send SIGINT  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
^C  
SIGINT received!  
Signal number = 2  
Program is running...  
Program is running...  
  
GNU nano 8.7.1 signal1.c *  
#include <stdio.h>  
#include <signal.h>  
#include <unistd.h>  
  
void handler(int sig) {  
    printf("\nSIGINT received!\n");  
    printf("Signal number = %d\n", sig);  
}  
  
int main() {  
    signal(SIGINT, handler);  
    while (1) {  
        printf("Program is running...\n");  
        sleep(2);  
    }  
    return 0;  
}
```

CODE 2:

```
praks@praks: ~  
GNU nano 8.7.1 sigchild.c *  
#include <stdio.h>  
#include <signal.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
void child_handler(int sig) {  
    printf("SIGCHLD received: Child has terminated\n");  
    wait(NULL);  
}  
  
int main() {  
    signal(SIGCHLD, child_handler);  
  
    pid_t pid = fork();  
  
    if (pid == 0) {  
        printf("Child is running...\n");  
        sleep(2);  
        printf("Child is exiting...\n");  
    } else {  
        printf("Parent is waiting...\n");  
        sleep(5);  
        printf("Parent finished.\n");  
    }  
    return 0;  
}  
  
praks@praks:~$ nano sigchild.c  
praks@praks:~$ gcc sigchild.c -o sigchild  
./sigchild  
\Parent is waiting...  
Child is running...  
Child is exiting...  
SIGCHLD received: Child has terminated  
Parent finished.
```

CODE 3:

```
praks@praks:~$ nano sigsegv.c
praks@praks:~$ gcc sigsegv.c -o sigsegv
./sigsegv
Pipe created.
Reading end closed.
Trying to write to the pipe...
praks@praks:~$ |
```



```
GNU nano 8.7.1 sigsegv.c *
#include <stdio.h>
#include <signal.h>
#include <stdlib.h>
#include <unistd.h>

void handle_sigsegv(int sig) {
    printf("\nSIGSEGV received!\n");
    printf("The reading end of the pipe is closed.\n");
    exit(1);
}

int main() {
    int fd[2];
    signal(SIGSEGV, handle_sigsegv);

    pipe(fd);
    printf("Pipe created.\n");

    // Close the reading end
    close(fd[0]);
    printf("Reading end closed.\n");

    printf("Trying to write to the pipe...\n");
    write(fd[1], "Hello", 5);
    printf("Write completed.\n");

    return 0;
}
```

CODE 4:

```
praks@praks: ~  
GNU nano 8.7.1 multi_signal.c *  
#include <stdio.h>  
#include <signal.h>  
#include <unistd.h>  
  
void handler(int sig) {  
    if (sig == SIGINT)  
        printf("\nSIGINT Received\n");  
    if (sig == SIGTERM)  
        printf("\nSIGTERM Received\n");  
    if (sig == SIGUSR1)  
        printf("\nSIGUSR1 Received\n");  
}  
  
int main() {  
    signal(SIGINT, handler);  
    signal(SIGTERM, handler);  
    signal(SIGUSR1, handler);  
  
    while (1) {  
        printf("Running...\n");  
        sleep(3);  
    }  
    return 0;  
}  
|  
  
praks@praks:~$ ./multi_signal &  
PID=$!  
echo "My program PID is: $PID"  
  
kill -SIGUSR1 $PID  
kill -SIGTERM $PID  
kill -SIGINT $PID  
[1] 833  
Running...  
My program PID is: 833
```

CODE 5:

```
praks@praks: ~  
praks@praks:~$ nano pipe_exec.c  
praks@praks:~$ gcc pipe_exec.c -o pipe_exec  
./pipe_exec  
drwxr-xr-x 2 praks praks 4096 Aug 4 05:33 Documents  
-rw-r--r-- 1 praks praks 248 Sep 30 10:07 FIFO_READ.c  
-rw-r--r-- 1 praks praks 287 Sep 30 10:08 FIFO_write.c  
-rw-r--r-- 1 praks praks 568 Sep 18 03:59 addtrans.c  
-rwxr-xr-x 1 praks praks 16168 Sep 18 03:35 adressspace  
-rw-r--r-- 1 praks praks 517 Sep 18 03:35 adressspace.c  
-rw-r--r-- 1 praks praks 533 Aug 11 04:00 b.c  
-rwxr-xr-x 1 praks praks 16032 Sep 29 03:51 cow  
-rw-r--r-- 1 praks praks 299 Sep 29 04:00 cow.c  
-rw-r--r-- 1 praks praks 424 Sep 18 03:56 demandpage.c  
-rwxr-xr-x 1 praks praks 15992 Aug 14 10:08 exec1  
-rw-r--r-- 1 praks praks 155 Aug 14 10:08 exec1.c  
-rw-r--r-- 1 praks praks 101 Aug 4 17:49 fork1.c  
-rwxr-xr-x 1 praks praks 16080 Aug 4 17:57 forkexec  
-rw-r--r-- 1 praks praks 259 Aug 4 17:57 forkexec.c  
-rw-r--r-- 1 praks praks 76 Aug 4 17:44 hello.c  
-rw-r--r-- 1 praks praks 382 Sep 18 04:00 memory.c  
-rwxr-xr-x 1 praks praks 16048 Sep 30 10:02 mkfifo_create  
-rw-r--r-- 1 praks praks 218 Sep 30 10:03 mkfifo_create.c  
-rwxr-xr-x 1 praks praks 16080 Oct 3 09:01 multi_signal  
-rw-r--r-- 1 praks praks 468 Oct 3 09:01 multi_signal.c  
-rwxr-xr-x 1 praks praks 16000 Aug 14 09:44 parentchild  
-rw-r--r-- 1 praks praks 226 Aug 14 09:43 parentchild.c  
-rw-r--r-- 1 praks praks 283 Aug 14 10:02 pids.c  
-rwxr-xr-x 1 praks praks 16216 Oct 3 09:10 pipe_exec  
-rw-r--r-- 1 praks praks 587 Oct 3 09:09 pipe_exec.c  
-rw-r--r-- 1 praks praks 68 Aug 4 03:50 praks.c  
-rwxr-xr-x 1 praks praks 16000 Aug 14 09:38 processid  
-rw-r--r-- 1 praks praks 108 Aug 14 09:38 processid.c  
-rw-r--r-- 1 praks praks 443 Sep 29 03:41 program7.c  
-rw-r--r-- 1 praks praks 240 Sep 29 03:41 program8.c  
-rw-r--r-- 1 praks praks 168 Sep 18 04:02 segmentationfault.c  
-rwxr-xr-x 1 praks praks 16160 Oct 3 09:00 sigchild  
-rw-r--r-- 1 praks praks 515 Oct 3 09:08 sigchild.c  
-rwxr-xr-x 1 praks praks 16112 Oct 3 08:58 signal1  
-rw-r--r-- 1 praks praks 305 Oct 3 08:57 signal1.c  
-rwxr-xr-x 1 praks praks 16256 Oct 3 09:00 sigsegv  
-rw-r--r-- 1 praks praks 546 Oct 3 09:00 sigsegv.c  
-rw-r--r-- 1 praks praks 301 Aug 24 05:57 skill2.c  
-rw-r--r-- 1 praks praks 991 Aug 7 09:22 task1.c  
-rw-r--r-- 1 praks praks 154 Aug 4 17:43 username.c  
-rw-r--r-- 1 praks praks 158 Sep 18 03:11 virtual.c  
-rw-r--r-- 1 praks praks 307 Aug 14 09:57 wait1.c  
-rw-r--r-- 1 praks praks 461 Sep 29 04:48 week3.c  
-rw-r--r-- 1 praks praks 635 Sep 29 04:04 week4.c  
praks@praks:~$ |
```

```
praks@praks: ~  
GNU nano 8.7.1 pipe_exec.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main() {  
    int fd[2];  
    pipe(fd);  
  
    if (fork() == 0) {  
        // Child 1: ls -l (writes to pipe)  
        dup2(fd[1], STDOUT_FILENO);  
        close(fd[0]);  
        close(fd[1]);  
        execlp("ls", "ls", "-l", NULL);  
    }  
  
    if (fork() == 0) {  
        // Child 2: grep ".c" (reads from pipe)  
        dup2(fd[0], STDIN_FILENO);  
        close(fd[0]);  
        close(fd[1]);  
        execlp("grep", "grep", ".c", NULL);  
    }  
  
    close(fd[0]);  
    close(fd[1]);  
  
    wait(NULL);  
    wait(NULL);  
  
    return 0;  
}
```